Introdução às Interfaces Gráficas em Python

Tkinter e CustomTkinter

Prof. Jelcimar Souza

IFES Cachoeiro de Itapemirim

14 de outubro de 2026

Roteiro da aula

1. Por que criar interfaces gráficas?
2. Tkinter: janela, widgets e layouts
3. Variáveis, eventos e caixas de diálogo
4. Projeto: calculadora de IMC com Tkinter
5. CustomTkinter: visual moderno
6. Projeto: calculadora de IMC com CustomTkinter
7. Boas práticas e exercícios

Objetivos da aula

Ao final desta aula você será capaz de:

- criar janelas com **Tkinter**;
- usar os widgets básicos (`Label`, `Button`, `Entry`...);
- organizar widgets com `pack`, `grid` e `place`;
- conectar botões a funções e tratar eventos;
- migrar para o **CustomTkinter**;
- construir um aplicativo completo.

Por que usar interface gráfica?

Terminal

- `input()` e `print()`
- Exige conhecer comandos
- Difícil para usuários leigos

Interface gráfica (GUI)

- Janelas, botões, menus
- Interação intuitiva
- Programas com aparência profissional

O que é o Tkinter?

- Biblioteca **padrão** do Python para GUI
- Baseada no toolkit Tcl/Tk
- Multiplataforma: Windows, Linux e macOS
- Não precisa de instalação adicional (na maioria dos sistemas)
- Importação usual:

```
import tkinter as tk
from tkinter import ttk, messagebox
```

Tkinter vs. CustomTkinter

	Tkinter	**CustomTkinter**
Instalação	Já vem com o Python	`pip install customtkinter`
Aparência	Clássica, simples	Moderna, temas claro/escuro
Base	Biblioteca padrão	Construída sobre o Tkinter
Sintaxe	`tk.Button(...)`	`ctk.CTkButton(...)`

A lógica é a mesma: aprender Tkinter primeiro facilita tudo.

Primeira janela

```
import tkinter as tk

janela = tk.Tk()                    # cria a janela principal
janela.title("Minha primeira janela")
janela.geometry("400x250")          # largura x altura

janela.mainloop()                   # mantém a janela aberta
```

O laço de eventos (`mainloop`)

1. Criar a janela principal
2. Criar os widgets
3. Posicionar os widgets (layout)
4. Chamar `mainloop()`
5. O programa **espera** cliques, teclas, etc.
6. Cada ação dispara uma função (*callback*)

Atenção: o código depois de `mainloop()` só executa quando a janela é fechada.

Widgets básicos

Widget	**Função**
`Label`	Exibe texto ou imagem
`Button`	Executa uma função ao ser clicado
`Entry`	Campo de texto de uma linha
`Text`	Área de texto com várias linhas
`Checkbutton`	Caixa de seleção
`Radiobutton`	Escolha de uma opção entre várias
`Listbox`	Lista de itens selecionáveis
`Frame`	Contêiner para agrupar widgets

Label e Entry

```python
import tkinter as tk

janela = tk.Tk()
janela.title("Olá, Tkinter")

rotulo = tk.Label(janela, text="Digite seu nome:", font=("Arial", 12))
rotulo.pack(pady=5)

campo = tk.Entry(janela, width=30)
campo.pack(pady=5)

def mostrar_valor():
    texto = campo.get()  # lê o texto digitado
    rotulo.config(text=f"Olá, {texto}!")

tk.Button(janela, text="Enviar", command=mostrar_valor).pack(pady=5)

janela.mainloop()
```

Button e o parâmetro command

```
def saudar():
    nome = campo.get()
    rotulo.config(text=f"Olá, {nome}!")

botao = tk.Button(janela, text="Saudar", command=saudar)
botao.pack(pady=10)

janela.mainloop()
```

Atenção

Use `command=saudar` **sem parênteses**. Com `saudar()` a função seria executada imediatamente, e não ao clicar.

Gerenciadores de layout

	Como organiza	**Quando usar**
`pack()`	Empilha widgets (cima/baixo ou lado a lado)	Layouts simples
`grid()`	Linhas e colunas, como uma tabela	Formulários
`place()`	Coordenadas absolutas (`x`, `y`)	Posicionamento fixo e específico

Layout com pack()

```
tk.Label(janela, text="Topo", bg="lightblue").pack(side="top", fill="x")
tk.Label(janela, text="Rodapé", bg="lightgreen").pack(side="bottom", fill="x")
tk.Button(janela, text="Esquerda").pack(side="left", padx=10)
tk.Button(janela, text="Direita").pack(side="right", padx=10)
```

Opções úteis: `side`, `fill`, `expand`, `padx`, `pady`.

Layout com grid()

```
tk.Label(janela, text="Peso (kg):").grid(row=0, column=0, padx=5, pady=5, sticky="e")
tk.Entry(janela).grid(row=0, column=1, padx=5, pady=5)

tk.Label(janela, text="Altura (m):").grid(row=1, column=0, padx=5, pady=5, sticky="e")
tk.Entry(janela).grid(row=1, column=1, padx=5, pady=5)

tk.Button(janela, text="OK").grid(row=2, column=0, columnspan=2, pady=10)
```

`sticky` alinha o widget na célula (`n`, `s`, `e`, `w`); `columnspan` ocupa várias colunas.

Layout com `place()` e regra de ouro

```
tk.Button(janela, text="OK").place(x=50, y=80)
tk.Label(janela, text="Centro").place(relx=0.5, rely=0.5, anchor="center")
```

Regra de ouro

Nunca misture `pack` e `grid` no mesmo contêiner. Para combinar layouts, use `Frame` diferentes.

Variáveis de controle

Ligam um widget a um valor: `StringVar`, `IntVar`, `DoubleVar`, `BooleanVar`.

```
texto = tk.StringVar(value="Olá")
tk.Label(janela, textvariable=texto).pack()
tk.Entry(janela, textvariable=texto).pack()

texto.set("Novo texto")     # atualiza o Label e o Entry
print(texto.get())          # lê o valor atual
```

Eventos com bind()

```
def tecla_pressionada(evento):
    print("Tecla:", evento.keysym)

janela.bind("<Key>", tecla_pressionada)
campo.bind("<Return>", lambda e: saudar())    # Enter no Entry
botao.bind("<Enter>", lambda e: print("Mouse sobre o botão"))
```

Eventos comuns: <Button-1> (clique), <Key>, <Return>, <Enter>, <Leave>.

Checkbutton e Radiobutton

```
var_ok = tk.BooleanVar()
tk.Checkbutton(janela, text="Aceito os termos", variable=var_ok).pack()

opcao = tk.StringVar(value="python")
tk.Radiobutton(janela, text="Python", variable=opcao, value="python").pack()
tk.Radiobutton(janela, text="Java", variable=opcao, value="java").pack()

print(var_ok.get(), opcao.get())
```

Caixas de diálogo (messagebox)

```python
from tkinter import messagebox

messagebox.showinfo("Título", "Operação concluída!")
messagebox.showwarning("Aviso", "Cuidado!")
messagebox.showerror("Erro", "Algo deu errado.")

resposta = messagebox.askyesno("Sair", "Deseja sair?")   # True ou False
```

Widgets temáticos: `ttk`

Visual mais próximo do sistema operacional: `ttk.Button`, `ttk.Entry`, `ttk.Combobox`, `ttk.Treeview`...

```
from tkinter import ttk

combo = ttk.Combobox(janela, values=["Python", "Java", "C"], state="readonly")
combo.current(0)
combo.pack(pady=10)

print(combo.get())
```

Projeto: calculadora de IMC – lógica

$$\text{IMC} = \frac{\text{peso}}{\text{altura}^2}$$

```
def calcular():
    try:
        peso = float(entry_peso.get().replace(",", "."))
        altura = float(entry_altura.get().replace(",", "."))

        if peso <= 0 or altura <= 0:
            raise ValueError

        imc = peso / altura ** 2
    except ValueError:
        messagebox.showerror(
            "Erro", "Digite peso e altura positivos e válidos."
        )
        return

    if imc < 18.5:
        classe = "Abaixo do peso"
    elif imc < 25:
```

Projeto: calculadora de IMC – interface

```
janela = tk.Tk()
janela.title("Calculadora de IMC")
janela.geometry("320x220")

tk.Label(janela, text="Peso (kg):").grid(row=0, column=0, padx=10, pady=10, sticky="e")
entry_peso = tk.Entry(janela);   entry_peso.grid(row=0, column=1)

tk.Label(janela, text="Altura (m):").grid(row=1, column=0, padx=10, pady=10, sticky="e")
entry_altura = tk.Entry(janela); entry_altura.grid(row=1, column=1)

tk.Button(janela, text="Calcular", command=calcular).grid(row=2, column=0, columnspan=2,
    pady=10)

resultado = tk.StringVar()
tk.Label(janela, textvariable=resultado, font=("Arial", 11, "bold")).grid(row=3, column=0,
    columnspan=2)

janela.mainloop()
```

CustomTkinter: instalação

- Biblioteca de terceiros, construída sobre o Tkinter
- Widgets modernos, cantos arredondados e temas claro/escuro
- Mesmos gerenciadores de layout: `pack`, `grid`, `place`

```
pip install customtkinter
```

```
import customtkinter as ctk
```

Primeira janela com CustomTkinter

```python
import customtkinter as ctk

ctk.set_appearance_mode("dark")          # "light", "dark" ou "system"
ctk.set_default_color_theme("blue")      # "blue", "green" ou "dark-blue"

app = ctk.CTk()
app.title("Minha primeira janela CTk")
app.geometry("400x250")

app.mainloop()
```

Equivalência de widgets

Tkinter	**CustomTkinter**
`Tk`	`CTk`
`Label`	`CTkLabel`
`Button`	`CTkButton`
`Entry`	`CTkEntry`
`Text`	`CTkTextbox`
`Checkbutton` / `Radiobutton`	`CTkCheckBox` / `CTkRadioButton`
`Frame`	`CTkFrame`
`ttk.Combobox`	`CTkComboBox`
—	`CTkSwitch`, `CTkSlider`, `CTkProgressBar`, `CTkTabview`

Widgets em ação com CustomTkinter

```
app = ctk.CTk()
app.geometry("360x300")

ctk.CTkLabel(app, text="Bem-vindo!", font=("Arial", 20, "bold")).pack(pady=15)

entrada = ctk.CTkEntry(app, placeholder_text="Seu nome", width=220)
entrada.pack(pady=5)

ctk.CTkButton(app, text="Enviar", corner_radius=20,
              command=lambda: print("Olá,", entrada.get())).pack(pady=10)

ctk.CTkSwitch(app, text="Receber notificações").pack(pady=5)
ctk.CTkSlider(app, from_=0, to=100).pack(pady=10)

app.mainloop()
```

Organização em classes

```
class App(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("App em classe")
        self.geometry("300x180")
        self.contador = 0

        self.label = ctk.CTkLabel(self, text="Cliques: 0", font=("Arial", 18))
        self.label.pack(pady=20)
        ctk.CTkButton(self, text="Clique aqui", command=self.incrementar).pack()

    def incrementar(self):
        self.contador += 1
        self.label.configure(text=f"Cliques: {self.contador}")

App().mainloop()
```

Dica: no CustomTkinter, altere propriedades com `.configure()`.

Projeto: IMC com CustomTkinter

```
class AppIMC(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("IMC")
        self.geometry("320x320")

        ctk.CTkLabel(self, text="Calculadora de IMC",
                     font=("Arial", 20, "bold")).pack
    (pady=15)
        self.peso = ctk.CTkEntry(self,
    placeholder_text="Peso (kg)")
        self.peso.pack(pady=8)
        self.altura = ctk.CTkEntry(self,
    placeholder_text="Altura (m)")
        self.altura.pack(pady=8)
        ctk.CTkButton(self, text="Calcular",
                      command=self.calcular).pack(
    pady=12)
        self.res = ctk.CTkLabel(self, text="", font=(
    "Arial", 14))
        self.res.pack()
```

```
    def calcular(self):
        try:
            p = float(self.peso.get().replace(
 ",", "."))
            a = float(self.altura.get().
 replace(",", "."))

            if p <= 0 or a <= 0:
                raise ValueError

            imc = p / a ** 2
        except ValueError:
            messagebox.showerror(
                "Erro", "Digite peso e altura
 positivos e válidos."
            )
            return

        if imc < 18.5:
            classe = "Abaixo do peso"
        elif imc < 25:
            classe = "Peso normal"
```

Boas práticas e erros comuns

- Esquecer o `mainloop()`: a janela abre e fecha na hora
- Misturar `pack` e `grid` no mesmo contêiner
- Escrever `command=funcao()` em vez de `command=funcao`
- `x = tk.Entry(...).pack()` faz `x` valer `None`
- Tarefas demoradas no botão congelam a janela (use `threading` ou `after()`)
- Separe a **lógica** da **interface**
- Use classes em aplicativos maiores

Exercícios e referências

Exercícios

1. Contador com botões “+1” e “Zerar”
2. Conversor Celsius → Fahrenheit
3. Lista de tarefas (adicionar e remover)
4. Tela de login (senha com `show="*"`)
5. Desafio: lista de tarefas com `CTkTabview`

Referências

- docs.python.org/3/library/tkinter.html
- customtkinter.tomschimansky.com
- github.com/TomSchimansky/CustomTkin

Obrigado!